

Chapter one

Fundamentals of Object-Oriented Programming

1. Introduction

With the advent of languages such as C, structured programming became very popular and was the paradigm of the 1980s. Structured programming proved to be a powerful tool that enabled programmers to write moderately complex programs fairly easily. However, as the program grows larger, even the structured approach failed to show the desired results in terms of bug –free, easy-to-maintain and reusable programs.

Object-Oriented Programming (**OOP**) is an approach to program organization and development, which attempts to eliminate some of the drawbacks of conventional programming methods by incorporating the best of structured programming features with several new advanced concepts. It is a new style of organizing and developing programs and has nothing to do with any particular. However, not all languages are suitable to implement the OOP concepts easily. Languages that support OOP features include **C++, C#, Java etc.** C++, an extension of C language, is basically a procedural language with object-oriented extension. The other popular language is **Java**, which is a pure object-oriented language.

2. The Object Oriented Paradigm

What is Object Oriented Orientation?

Basically, OOP is a technique of modeling some kind of system in software based Objects.

- It represents a fundamentally different way of thinking about a problem solving than other software development methodologies.
- It enables us to reduce the software systems **complexity** and/or failure.

A research made by Standish Group in 1985 says,

- On average only 16.2% of projects are completed on time and with budget.
- 31.1% of the projects were cancelled before they ever get completed.
- 52.7% of the projects will cost 189% of their original estimate.

OOP treats **data** as a critical element in the program development and does not allow it to flow freely around the system. It ties data more closely to the functions that operate on it and protects it from unintentional modification by other functions. OOP allows us to decompose a problem into a number of entities called **Objects** and then build data and functions (known as Methods in java) around these entities.

The combination of data and methods make up an object (see **Fig 1.1**)

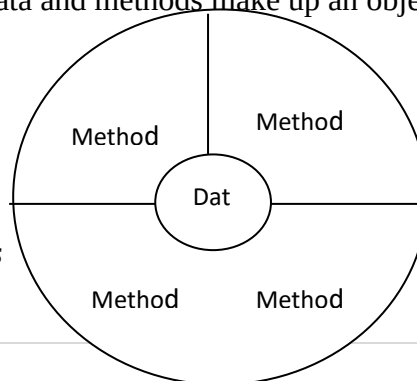


Fig 1.1 *Object=data + Methods*

Object Oriented programming

The data of an object can be accessed only by the methods associated with that object. However, methods of one object can access the methods of other objects.

1.1 Features of Object Oriented paradigm (standard)

- ✚ Programs are divided into
 - Interaction of what are known as objects.
 - Methods that operate on the data of an object are tied together in the data structure.
- ✚ Data is hidden and cannot be accessed by external functions.
- ✚ Emphasis is on data rather than procedures/processes.
- ✚ Objects may communicate with each other through methods.
- ✚ Data structures are designed such that they characterize the objects.
- ✚ New data and methods can be added whenever necessary.
- ✚ Follows bottom-up approach in program design.

Definition:

OOP: - is an approach that provides a way of modularizing programs by creating partitioned memory area for both data and methods that can be used as templates for creating copies of such modules on demand. This means that an object is considered to be a partitioned area of computer memory that stores data and a set of operations that can operate on those data. Since the memory partitions are independent, the objects can be used in a variety of different programs without modification.

1.2 Basic Concepts of Object Oriented Programming

✚ **Object**

An object is a core concept, and is a model or representation of real-world entity or logical idea or concept. It may represent a person, a place, a bank account, a table of data or any item that the program may handle. It may also represent user-defined data types such as vectors and lists. As pointed out earlier, an object takes up space in the memory and has an associated address like a structure in C.

When a program is executed, the objects interact by sending messages to one another. For example let's say 'customer' and 'account' are two objects in banking program, then the customer object may send a message to the account object requesting for the balance. Each object contains data and code to manipulate the data. Objects can interact without having to know the details of each other's data or code. It is sufficient to know the type of message accepted and the type of response returned by the objects.

✚ **Class**

We just mentioned that objects contain data and code to manipulate that data. The entire set of data and code of an object can be made a user-defined data type using concept of a *class*. A class may be thought of as a 'data type' and an object as a 'variable' of that data type. Once a class has been defined, we can create any number of objects belonging to that class. Each object is associated with the data of type class with which they are created. A class is thus a collection of objects of similar type. For Example mango, apple and orange are members of the class fruit.

Object Oriented programming

Definition: A class is a template used to create multiple objects with similar features.

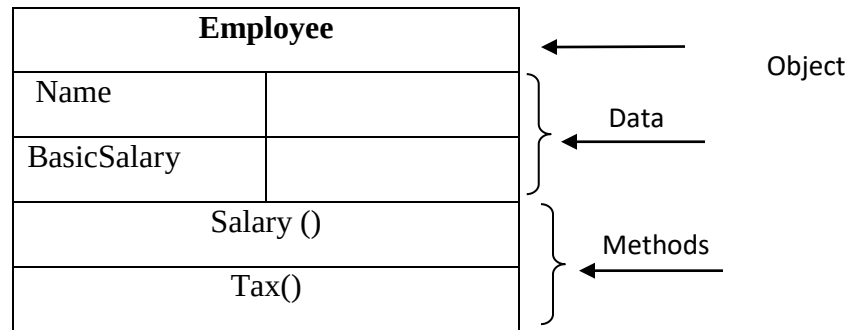


Fig 1.2 Representation of an Object

Classes are user-defined data types and behave like the built-in data types of a programming language.

Data Abstraction and Encapsulation

The wrapping (packaging) up of data and methods into a single unit (called class) is known as *encapsulation*. Data encapsulation is the most remarkable feature of a class. The data is not accessible to the outside world and only those methods, which are wrapped in the class, can access it. This insulation (protecting) of the data from direct access by the program is called *data hiding*.

Abstraction:-refers to the act of representing essential features without including the background details or explanations. Classes use the concept of abstraction and are defined as a list of abstract attributes such as size, weight, and cost and methods that operate on these attributes. They encapsulate all essential properties of the objects that are to be created.

Inheritance

Inheritance is the process by which objects of one class acquire the properties of objects of another class. Inheritance supports the concept of hierarchical classification. In OOP, the concept of inheritance provides the idea of reusability. This means that we can add additional features to an existing class without modifying it. This is possible by deriving a new class from an existing one. The new class will have the combined features of both the classes. Thus the real appeal and power of the inheritance mechanism is that it allows the programmer to reuse a class that is almost, but not exactly, what he wants, and to tailor the class in such a way that it does not introduce any undesirable side effects into the rest of the classes.

Polymorphism

Polymorphism means the ability to take more than one form. For example, an operation may exhibit different behavior in different instances. The behavior depends on the type of data used in the operation.

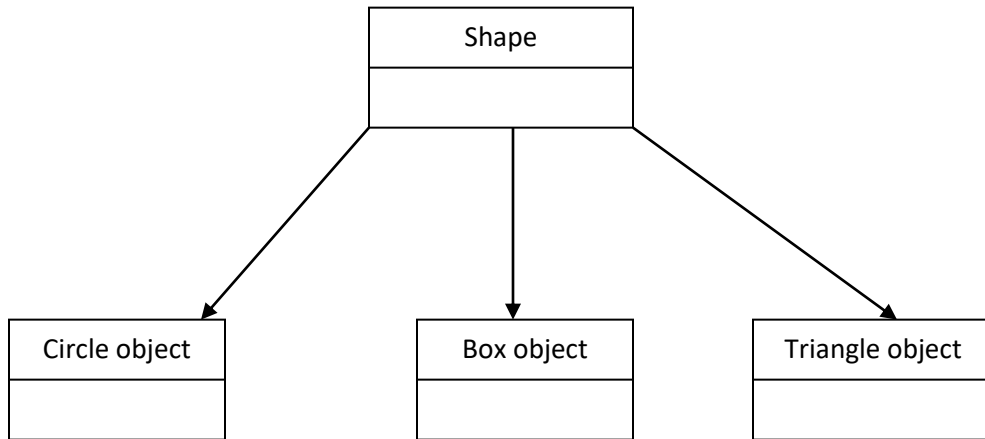


Fig 1.3 *polymorphism*

Polymorphism plays an important role in allowing objects having different internal structure to share the same external interface.

Dynamic Binding

It refers to the linking of a procedure call to the code to be executed in response to the call. Dynamic binding means that the code associated with a given procedure call is not known until the time of the call at run time. It is associated with polymorphism and inheritance. A procedure call associated with a polymorphic reference depends on the dynamic type of that reference.

Consider the procedure ‘draw’ in **Fig 1.3** above. By inheritance, every object will have this procedure. Its algorithm is, however, unique to each object and so the draw procedure will be redefined in each class that defines the object. At run-time, the code matching the object under current reference will be called.

Message Communication

An object-oriented program consists of a set of objects that communicate with each other. The process of programming in object-oriented language, therefore, involves the following three basic steps:

1. Creating classes that define objects and their behavior.
2. Creating objects from class definitions.
3. Establishing communication among objects.

Objects communicate with one another by sending and receiving information that matches the same way as people pass messages to one another. A message for an object is a request for execution of a procedure, and therefore will invoke a method (procedure) in the receiving object that generates the desired result as shown in **Fig 1.4** below.

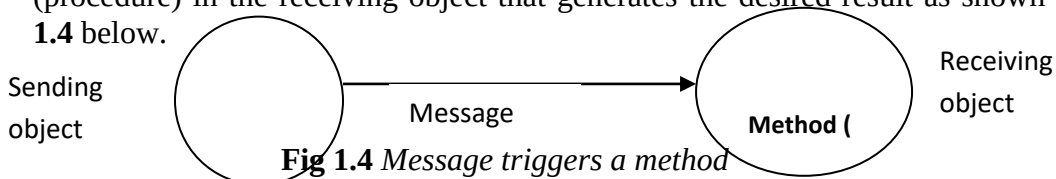


Fig 1.4 *Message triggers a method*

Object Oriented programming

Message passing involves specifying the name of the object, the name of the method (message) and the information to be sent. For example consider the statement ***Employee.Salary (name);***

Here, Employee is the object, Salary is the message and name is the parameter that contains information.

Note: - Objects have life-cycle. They can be created and destroyed. Communication with an object is feasible as long as it is alive.

1.3 Benefits of Object Oriented Programming

The following are some of the advantages of OOP:

Through inheritance, we can eliminate redundant code and extend the use of existing classes.

- We can build programs from the standard working modules that communicate with one another, rather than having to start writing the code from scratch. This leads to saving of development time and higher productivity.
- ❖ The principle of data hiding helps the programmer to build secure programs that cannot be invaded by code in other parts of the program.
- ❖ It is possible to have multiple objects to coexist without any interference.
- ❖ It is easy to partition the work in a project based on objects/reducing complexity.
- ❖ Object-oriented systems can be easily upgraded from small to large systems.
- ❖ Message passing techniques for communication between objects make the interface descriptions with the external systems much simpler.
- ❖ Software complexity can be easily managed.

1.4 Applications of OOP

The promising areas for application of OOP include:

- Simulation and Modeling
- Object-oriented database
- Hypertext and hypermedia
- AI and expert systems.
- Real-time systems
- Decision support and office automation systems.

Overview of Java Language

What is java?

Java is a high level, robust, secured and object-oriented programming language. Java was developed by a **Green team** led by James Gosling at **SUN Microsystems**. Originally called **Oak**, and was designed in 1991 for use in embedded consumer electronic appliances. In 1995, renamed as **Java** and was redesigned for developing Internet applications.

Java has become enormously popular. Java's rapid rise and wide acceptance can be traced to its design characteristics, particularly its promise that you can ***write a program once and run it anywhere*** as in fig2.1. As stated in the Java language white paper by **SUN**, Java is simple,

Object Oriented programming

object-oriented, distributed, interpreted, robust, secure, architecture-neutral, portable, high-performance, multithreaded, and dynamic.

Java is a full-featured, general-purpose programming language that is capable of developing robust mission-critical applications. Today, it is used not only for Web programming, but also for developing standalone applications across platforms on servers, desktops, and mobile devices as in fig 2.2. It was used to develop the code to communicate with and control the robotic rover that rolled on Mars. Many companies that once considered Java to be more hype than substance are now using it to create distributed applications accessed by customers and partners across the Internet. For every new project being developed today, companies are asking how they can use Java to make their work easier.

Java is popular

- In the **TOP 3** (if not the most popular)
- **3 billion** devices run...
- **Platform independency** (write once, run anywhere)
- Lots of available, very **well paying jobs**
 - **Great demand** for Java developers
- **Wide usability**



Fig 2.1 java trade mark

Java as a platform has three category/editions **In Application Program Interface (API)** as can be observed in the following tree diagram in fig 2.2 below. The three editions of Java are:-

1. Java Micro Edition (ME) for mobile apps development.
2. Java Standard Edition (SE) for developing standalone application like desktop apps, gaming apps etc.
3. Java Enterprise Edition (EE) for large scale apps like core banking

Java is a full-fledged and powerful language that can be used in many ways

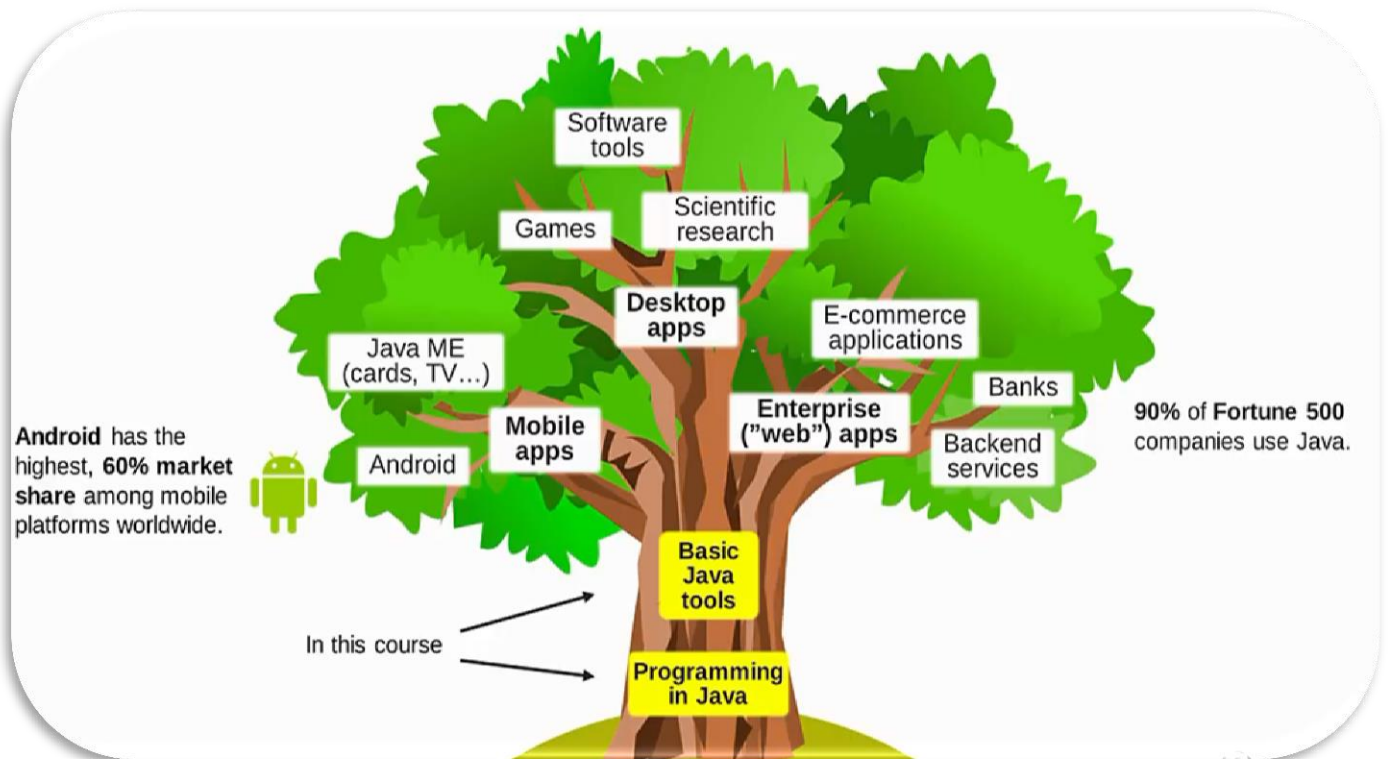


Fig 2.2 the core java editions

Why Java?

The main objective of **Green team/Java team** was to improve C++ compiler and make the execution of a C++ source code **platform independent**. The motto of the team was ***"Write Once and Run anywhere"*** for all computer programs. As a result, the team has come up with the concept of **Java Virtual Machine (JVM)** which we will discuss in the following sections. For demonstration a sample program will be considered.

A simple Java Program

Every Java program must have at least one class. A class is a construct that defines data and methods. Each class has a name. By convention, class names start with an uppercase letter. In this example, the class name is **Welcome**.

```
public class Welcome {  
    public static void main(String[] args){  
        System.out.println("welcome to java");  
    } }  
}
```

In order to run a class, the class must contain a method named **main**. The JVM executes the program by invoking the **main method**.

Object Oriented programming

A method is a construct that contains statements. The **main** method in this program contains the **System.out.println** statement. This statement prints a message "**Welcome to Java!**" to the console.

Note: You are probably wondering about such points as why the **main** method is declared this way and why **System.out.println(...)** is used to display a message to the console. Your questions cannot be fully answered yet. For the time being, you will just have to accept that this is how things are done. You will find the answers in the coming chapters.

Like any other programming language, Java has its own syntax, and you need to write code that obeys the syntax rules. The Java compiler will report syntax errors if your program violates the syntax rules. How every java program is compiled and executed will be discussed in the following section.

Java Virtual Machine

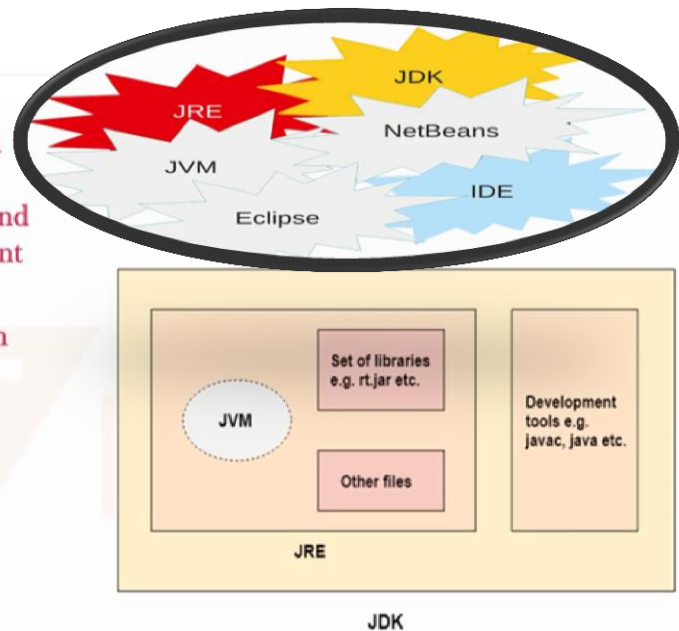
In java, the acronyms in the figure below are very common. In this section, the fundamental concepts related to them will be discussed.

JDK is an acronym for Java Development Kit. The Java Development Kit (JDK) is a software development environment which is used to develop Java applications and applets. It physically exists. It contains JRE + development tools.

JDK is an implementation of any one of the below given Java Platforms released by Oracle Corporation:

- Standard Edition Java Platform
- Enterprise Edition Java Platform
- Micro Edition Java Platform

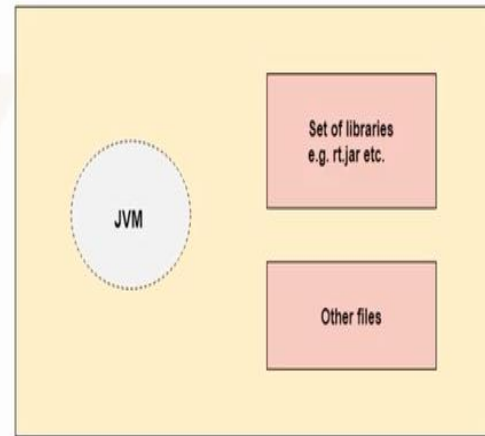
The JDK contains a private Java Virtual Machine (JVM) and a few other resources such as an interpreter/loader (java), a compiler (javac), an archiver (jar), a documentation generator (Javadoc), etc. to complete the development of a Java Application.



Object Oriented programming

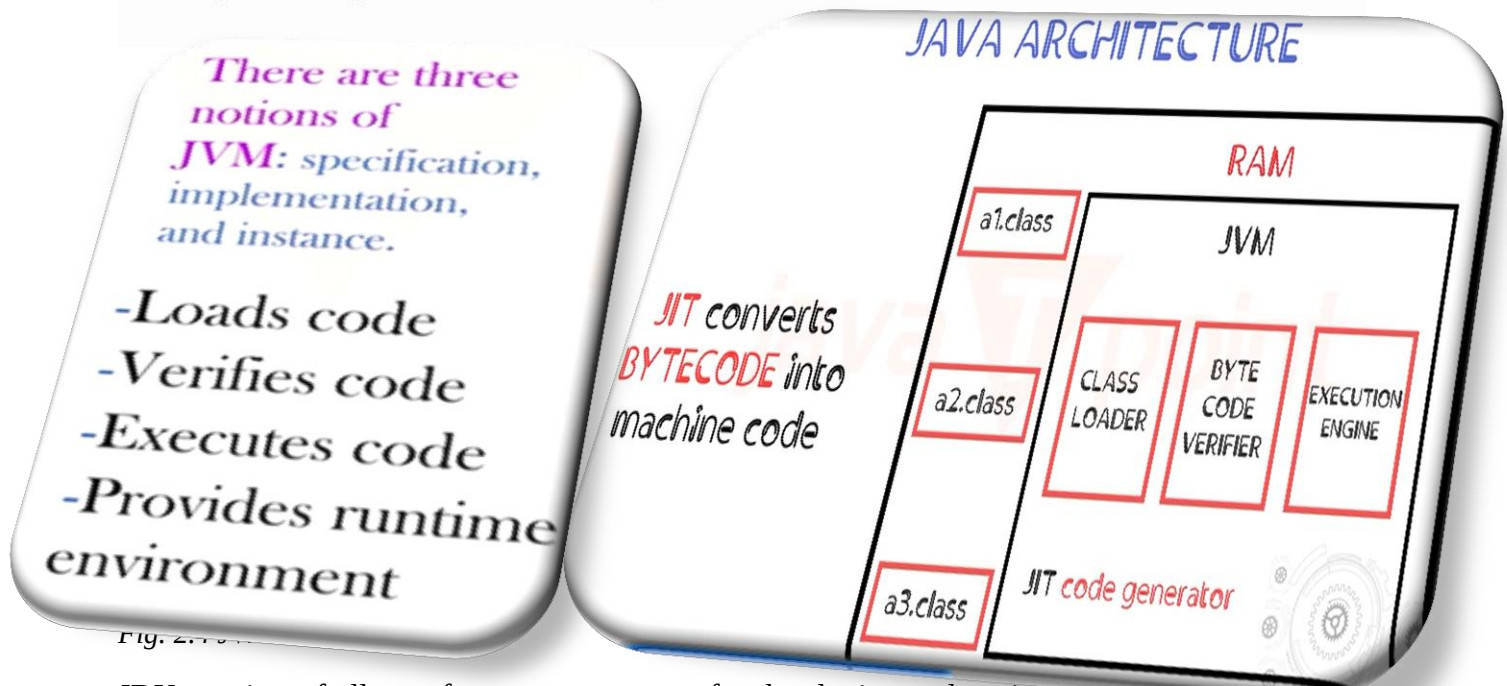
Java Virtual Machine.....

JRE is an acronym for Java Runtime Environment. It is also written as Java RTE. The Java Runtime Environment is a set of software tools which are used for developing Java applications. It is used to provide the runtime environment. It is the implementation of JVM. It physically exists. It contains a set of libraries + other files that JVM uses at runtime.



JRE

The implementation of JVM is also actively released by other companies besides Sun Micro Systems.



JDK consists of all set of separate programs for developing and testing Java programs, which is invoked from a command line. Besides JDK, there are more than a dozen Java development tools **Integrated Development Environment (IDE)** on the market today. Some development tools/IDE are:

- NetBeans Open Source by Sun (<http://www.netbeans.org/>)
- Eclipse Open Source by IBM (<http://www.eclipse.org/>)

Object Oriented programming

These IDEs are bundled with the corresponding JDK. A Java development tool is software that provides an *integrated development environment (IDE)* for rapidly developing Java programs. Editing, compiling, building, debugging, and online help are integrated in one graphical user interface. Just enter source code in one window or open an existing file in a window, then click a button, menu item, or function key to compile and run the program.

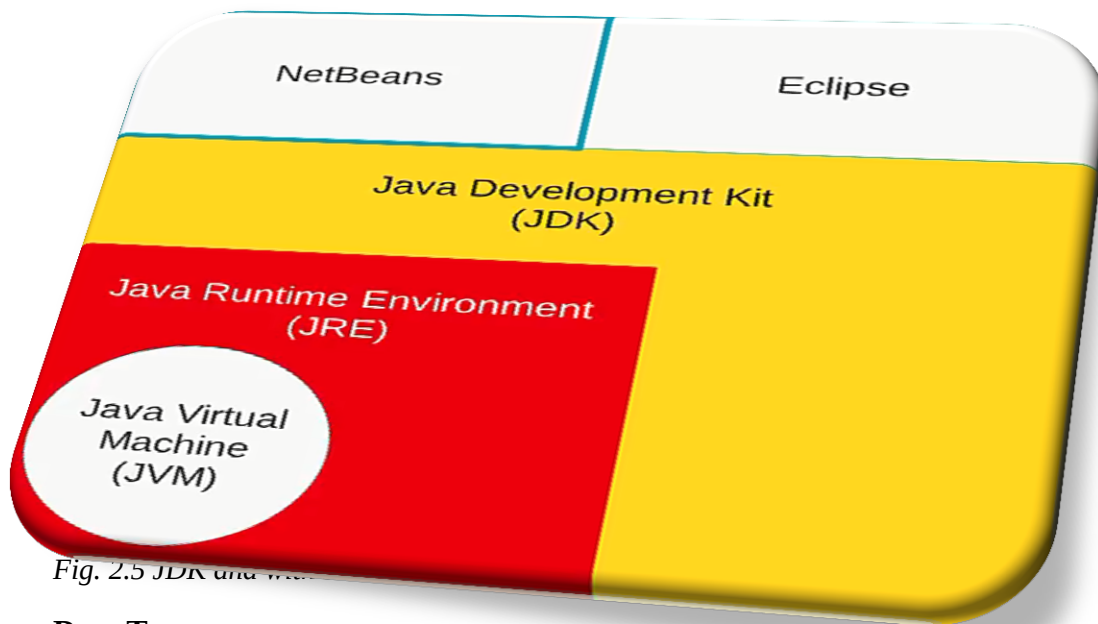


Fig. 2.5 JDK and IDEs

Data Types

Variables are nothing but reserved memory locations to store values. This means that when you create a variable you reserve some space in memory. Based on the data type of a variable, the operating system allocates memory and decides what can be stored in the reserved memory. Therefore, by assigning different data types to variables, you can store integers, decimals, or characters in these variables.

There are two data types available in Java:-

1. Primitive Data Types
2. Reference/Object Data Types

1. Primitive Data Types

There are eight primitive data types supported by Java. Primitive data types are predefined by the language and named by a keyword. Let's now look into detail about the eight primitive data types.

1. byte:

Byte data type is an 8-bit signed two's complement integer.

- The range of signed value from -2^7 (-128) to $2^7 - 1$ (127)
- Default value is 0
- Byte data type is used to save space in large arrays, mainly in place of integers, since a byte is four times smaller than an int.

Example: byte a = 100, byte b = -50

2. Short:

Short data type is a 16-bit signed two's complement integer.

- The range of signed value from **-32,768** $= -2^{15}$ to **32,767 inclusive** $2^{15} - 1$
- Short data type can also be used to save memory as byte data type.
- A short is 2 times smaller than an **int**
- Default value is 0.

Example: short s = 10000, short r = -20000

3. int:

int data type is a 32-bit signed two's complement integer.

- Minimum value is - 2,147,483,648. -2^{31}
- Maximum value is 2,147,483,647 inclusive. $2^{31} - 1$
- **int** is generally used as the default data type for integral values unless there is a concern about memory.
- The default value is 0.

Example: **int a = 100000, int b = -200000**

4. long:

Long data type is a 64-bit signed two's complement integer.

- Minimum value is -9,223,372,036,854,775,808. -2^{63}
- Maximum value is 9,223,372,036,854,775,807 inclusive. $2^{63} - 1$
- This type is used when a wider range than int is needed.
- Default value is 0L.

Example: long a = 100000L, long b = -200000L

5. float:

Float data type is a single-precision 32-bit IEEE 754 floating point.

- Float is mainly used to save memory in large arrays of floating point numbers.
- Default value is 0.0f.
- Float data type is never used for precise values such as currency.

Example: float f1 = 234.5f

6. double:

Double data type is a double-precision 64-bit IEEE 754 floating point. This data type is generally used as the default data type for decimal values, generally the default choice. Double data type should never be used for precise values such as currency.

Default value is 0.0d.

Example: double d1 = 123.4

7. boolean:

Boolean data type represents one bit of information. There are only two possible values: true and false.

This data type is used for simple flags that track true/false conditions.

Default value is false.

Example: boolean one = true

8. char:

char data type is a single 16-bit Unicode character. Minimum value is '\u0000' or 0.

Maximum value is '\uffff' or 65, 535 inclusive. Char data type is used to store any character.

Example: char letterA = 'A'

2. Reference Data Types:

Reference variables are created using defined constructors of the classes. They are used to access objects. These variables are declared to be of a specific type that cannot be changed. Class objects, and various types of array variables come under reference data type. Default value of

any reference variable is null. A reference variable can be used to refer to any object of the declared type or any compatible type.

Java Literals

A literal is a source code representation of a fixed value. They are represented directly in the code without any computation. Literals can be assigned to any primitive type variable. For example:

```
byte a = 68;  
char a = 'A'
```

byte, int, long, and short can be expressed in decimalbase10, hexadecimalbase16 or octalbase8 number systems as well. Prefix 0 is used to indicate octal and prefix 0x indicates hexadecimal when using these number systems for literals. For example:

```
intdecim al = 100;  
intoctal = 0144;  
inhexa= 0x64;
```

String literals in Java are specified like they are in most other languages by enclosing a sequence of characters between a pair of double quotes. Examples of string literals are:

```
"Hello World"  
"two\nlines"  
 "\"This is in quotes\""
```

String and char types of literals can contain any Unicode characters. For example:

```
chara = "\u0001";  
String a = "\u0001";
```

Java escape sequences

Java language supports few special escape sequences for String and char literals as well. They are:

Notation	Character represented
<code>\n</code>	Newline 0x0a
<code>\r</code>	Carriage return 0x0d
<code>\f</code>	Formfeed 0x0c
<code>\b</code>	Backspace 0x08
<code>\s</code>	Space 0x20
<code>\t</code>	tab
<code>\"</code>	Double quote
<code>\'</code>	Single quote
<code>\\</code>	backslash
<code>\ddd</code>	Octal character ddd
<code>\uxxxx</code>	Hexadecimal UNICODE character xxxx

<code>\n</code>	Newline 0x0a
<code>\r</code>	Carriage return 0x0d
<code>\f</code>	Formfeed 0x0c
<code>\b</code>	Backspace 0x08
<code>\s</code>	Space 0x20
<code>\t</code>	tab
<code>\"</code>	Double quote
<code>\'</code>	Single quote
<code>\\</code>	backslash
<code>\ddd</code>	Octal character ddd
<code>\uxxxx</code>	Hexadecimal UNICODE character xxxx

Declaration of Variables

A variable provides us with named storage that our programs can manipulate. Each variable in Java has a specific type, which determines the size and layout of the variable's memory; the range of values that can be stored within that memory; and the set of operations that can be applied to the variable. We must declare all variables before they can be used.

Following is the basic form of a variable declaration –

data type variable [= value][, variable [= value] ...] ;

Here *data type* is one of Java's datatypes and *variable* is the name of the variable. To declare more than one variable of the specified type, you can use a comma-separated list. Following are valid examples of variable declaration and initialization in Java.

Example

```
int a, b, c; // Declares three ints, a, b, and c.
int a =10, b =10; // Example of initialization
byte B =22; // initializes a byte type variable B.
double pi =3.14159; // declares and assigns a value of PI.
char a ='a'; // the char variable a is initialized with value 'a'
```

There are three kinds of variables in Java

1. Local variables
2. Instance variables
3. Class/Static variables

Local Variables

- Local variables are declared in methods, constructors, or blocks.
- Local variables are created when the method, constructor or block is entered and the variable will be destroyed once it exits the method, constructor, or block.
- Access modifiers cannot be used for local variables.
- Local variables are visible only within the declared method, constructor, or block.
- Local variables are implemented at stack level internally.
- There is no default value for local variables, so local variables should be declared and an initial value should be assigned before the first use.

Example

Here, *age* is a local variable. This is defined inside *pupAge()* method and its scope is limited to only this method.

```
public class Test{
    public void pupAge(){
        int age =0; // initialization
        age = age +7;
        System.out.println("Puppy age is : "+ age);
    }
    public static void main(String args[]){
        Test test = new Test();
        test.pupAge();
    }
}
```

This will produce the following result –

Output **Puppy age is: 7**

Example

Following example uses *age* without initializing it, so it would give an error at the time of compilation.

```
public class Test {  
    public void pupAge() {  
        int age;  
        age = age + 7;  
        System.out.println("Puppy age is : " + age);  
    }  
    public static void main(String[] args) {  
        Test test = new Test();  
        test.pupAge();  
    }  
}
```

This will produce the following error while compiling it

Output

Test.java:4:variable number might not have been initialized

age = age + 7;

1 error

Operators

Java provides a rich set of operators to manipulate variables. We can divide all the Java operators into the following groups

- ❖ Arithmetic Operators
- ❖ Relational Operators
- ❖ Bitwise Operators
- ❖ Logical Operators
- ❖ Assignment Operators
- ❖ Miscellaneous Operators

The Arithmetic Operators

Arithmetic operators are used in mathematical expressions in the same way that they are used in algebra. The following table lists the arithmetic operators. Assume integer variable A holds 10 and variable B holds 20, then –

Operator	Description	Example
+ (Addition)	Adds values on either side of the operator.	A + B will give 30
- (Subtraction)	Subtracts right-hand operand from left-hand operand.	A - B will give -10
* (Multiplication)	Multiplies values on either side of the operator.	A * B will give 200

Object Oriented programming

/ (Division)	Divides left-hand operand by right-hand operand.	B / A will give 2
% (Modulus)	Divides left-hand operand by right-hand operand and returns remainder.	B % A will give 0
++ (Increment)	Increases the value of operand by 1.	B++ gives 21
-- (Decrement)	Decreases the value of operand by 1.	B-- gives 19

The Relational Operators

There are following relational operators supported by Java language. Assume variable A holds 10 and variable B holds 20, then

Operator	Description	Example
== (equal to)	Checks if the values of two operands are equal or not, if yes then condition becomes true.	(A == B) is not true.
!= (not equal to)	Checks if the values of two operands are equal or not, if values are not equal then condition becomes true.	(A != B) is true.
> (greater than)	Checks if the value of left operand is greater than the value of right operand, if yes then condition becomes true.	(A > B) is not true.
< (less than)	Checks if the value of left operand is less than the value of right operand, if yes then condition becomes true.	(A < B) is true.
>= (greater than or equal to)	Checks if the value of left operand is greater than or equal to the value of right operand, if yes then condition becomes true.	(A >= B) is not true.
<= (less than or equal to)	Checks if the value of left operand is less than or equal to the value of right operand, if yes then condition becomes true.	(A <= B) is true.

The Bitwise Operators

Java defines several bitwise operators, which can be applied to the integer types, long, int, short, char, and byte. Bitwise operator works on bits and performs bit-by-bit operation. Assume if a = 60 and b = 13; now in binary format they will be as follows

a = 0011 1100

b = 0000 1101

a&b = 0000 1100

a|b = 0011 1101

a^b = 0011 0001

Object Oriented programming

$\sim a = 1100\ 0011$

The following table lists the bitwise operators

Assume integer variable A holds 60 and variable B holds 13 then

Operator	Description	Example
& (bitwise and)	Binary AND Operator copies a bit to the result if it exists in both operands.	(A & B) will give 12 which is 0000 1100
(bitwise or)	Binary OR Operator copies a bit if it exists in either operand.	(A B) will give 61 which is 0011 1101
^ (bitwise XOR)	Binary XOR Operator copies the bit if it is set in one operand but not both.	(A ^ B) will give 49 which is 0011 0001
~ (bitwise compliment)	Binary Ones Complement Operator is unary and has the effect of 'flipping' bits.	(~A) will give -61 which is 1100 0011 in 2's complement form due to a signed binary number.
<< (left shift)	Binary Left Shift Operator. The left operands value is moved left by the number of bits specified by the right operand.	A << 2 will give 240 which is 1111 0000
>> (right shift)	Binary Right Shift Operator. The left operands value is moved right by the number of bits specified by the right operand.	A >> 2 will give 15 which is 1111
>>> (zero fill right shift)	Shift right zero fill operator. The left operands value is moved right by the number of bits specified by the right operand and shifted values are filled up with zeros.	A >>>2 will give 15 which is 0000 1111

The Logical Operators

The following table lists the logical operators. Assume Boolean variables A holds true and variable B holds false, then

Operator	Description	Example
&& (logical and)	Called Logical AND operator. If both the operands are non-zero, then the condition becomes true.	(A && B) is false
(logical or)	Called Logical OR Operator. If any of the two operands are non-zero, then the condition becomes true.	(A B) is true

Object Oriented programming

! (logical not)	Called Logical NOT Operator. Use to reverses the logical state of its operand. If a condition is true then Logical NOT operator will make false.	!(A && B) is true
-----------------	--	-------------------

The Assignment Operators

Following are the assignment operators supported by Java language

Operator	Description	Example
=	Simple assignment operator. Assigns values from right side operands to left side operand.	$C = A + B$ will assign value of $A + B$ into C
+=	Add AND assignment operator. It adds right operand to the left operand and assign the result to left operand.	$C += A$ is equivalent to $C = C + A$
-=	Subtract AND assignment operator. It subtracts right operand from the left operand and assign the result to left operand.	$C -= A$ is equivalent to $C = C - A$
*=	Multiply AND assignment operator. It multiplies right operand with the left operand and assign the result to left operand.	$C *= A$ is equivalent to $C = C * A$
/=	Divide AND assignment operator. It divides left operand with the right operand and assign the result to left operand.	$C /= A$ is equivalent to $C = C / A$
%=	Modulus AND assignment operator. It takes modulus using two operands and assign the result to left operand.	$C \% = A$ is equivalent to $C = C \% A$
<<=	Left shift AND assignment operator.	$C << = 2$ is same as $C = C << 2$
>>=	Right shift AND assignment operator.	$C >> = 2$ is same as $C = C >> 2$
&=	Bitwise AND assignment operator.	$C \& = 2$ is same as $C = C \& 2$
^=	bitwise exclusive OR and assignment operator.	$C \wedge = 2$ is same as $C = C \wedge 2$
=	bitwise inclusive OR and assignment operator.	$C = 2$ is same as $C = C 2$

Miscellaneous Operators

There are few other operators supported by Java Language.

Conditional Operator (? :)

Conditional operator is also known as the **ternary operator**. This operator consists of three operands and is used to evaluate Boolean expressions. The goal of the operator is to decide, which value should be assigned to the variable. The operator is written as –

variable x = (expression) ? value if true : value if false

Following is an example –

Example

```
Public class Test{  
  
public static void main(String args[]){  
int a, b;  
    a =10;  
    b =(a ==1)?20:30;  
System.out.println("Value of b is : "+ b );  
  
    b =(a ==10)?20:30;  
System.out.println("Value of b is : "+ b );  
}  
}
```

This will produce the following result –

Output

Value of b is : 30

Value of b is : 20

Precedence of Java Operators

Operator precedence determines the grouping of terms in an expression. This affects how an expression is evaluated. Certain operators have higher precedence than others; for example, the multiplication operator has higher precedence than the addition operator

For example, x = 7 + 3 * 2; here x is assigned 13, not 20 because operator * has higher precedence than +, so it first gets multiplied with 3 * 2 and then adds into 7. Here, operators with the highest precedence appear at the top of the table, those with the lowest appear at the bottom. Within an expression, higher precedence operators will be evaluated first.

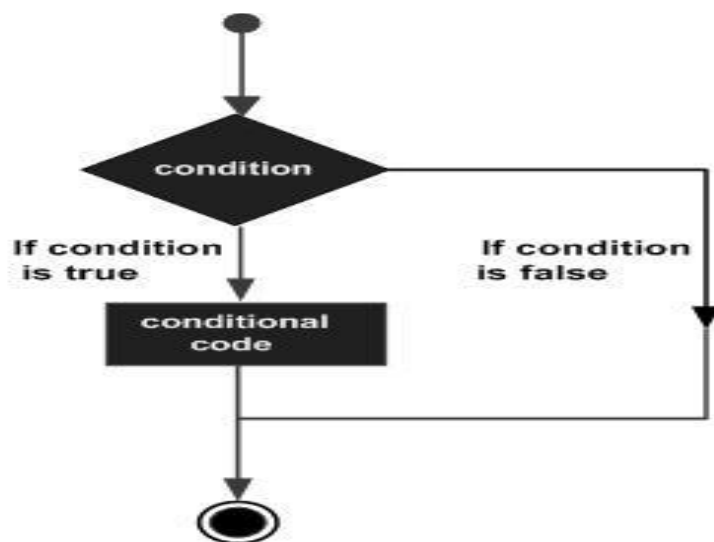
Category	Operator	Associativity
Postfix	expression++ expression--	Left to right
Unary	++expression --expression +expression -expression ~ !	Right to left
Multiplicative	* / %	Left to right

Object Oriented programming

Additive	+ -	Left to right
Shift	<<>>>>	Left to right
Relational	<><= >= instanceof	Left to right
Equality	== !=	Left to right
Bitwise AND	&	Left to right
Bitwise XOR	^	Left to right
Bitwise OR		Left to right
Logical AND	&&	Left to right
Logical OR		Left to right
Conditional	?:	Right to left
Assignment	= += -= *= /= %= ^= = <<= >>= >>>=	Right to left

Control Flow

Decision making structures have one or more conditions to be evaluated or tested by the program, along with a statement or statements that are to be executed if the condition is determined to be true, and optionally, other statements to be executed if the condition is determined to be false. Following is the general form of a typical decision making structure found in most of the programming languages



Java programming language provides following types of decision making statements

if Condition

An **if** statement consists of a Boolean expression followed by one or more statements.

Syntax

Following is the syntax of an if statement

```
if(Boolean_expression)
{
    // Statements will execute if the Boolean expression is true
}
```

If the Boolean expression evaluates to **true** then the block of code inside the **if** statement will be executed. If not, the first set of code after the end of the **if** statement (after the closing curly brace) will be executed.

```
public class Test {

    public static void main(String args[]) {
        int x = 10;

        if (x < 20) {
            System.out.print("This is if statement");
        }
    }
}
```

This will produce the following result

Output *This is if statement.*

if...else statement

An **if** statement can be followed by an optional **else** statement, which executes when the Boolean expression is false.

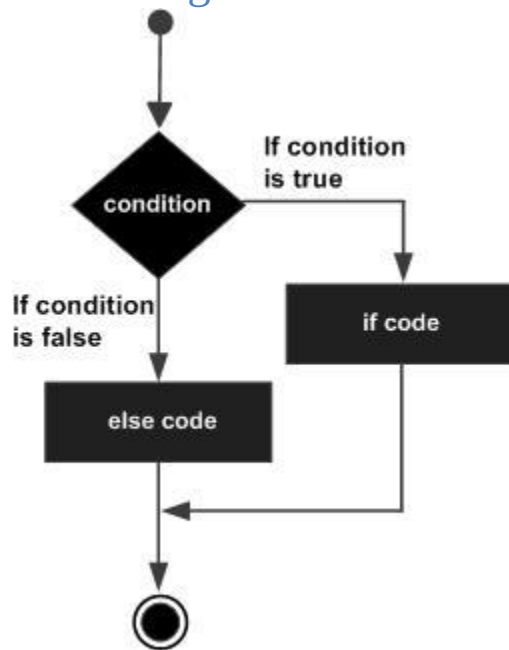
Syntax

Following is the syntax of an if...else statement –

```
if(Boolean_expression) {
    // Executes when the Boolean expression is true
}
else {
    // Executes when the Boolean expression is false
}
```

If the **boolean** expression evaluates to true, then the if block of code will be executed, otherwise else block of code will be executed.

Flow Diagram



Example

```
Public class Test{  
  
    Public static void main(String args[]){  
        int x =30;  
  
        if( x <20){  
            System.out.print("This is if statement");  
        }else{  
            System.out.print("This is else statement");  
        }  
    }  
}
```

This will produce the following result –

Output This is else statement

The if...else if...else Statement

An *if* statement can be followed by an optional ***else if...else statement***, which is very useful to test various conditions using single if...else if statement.

When using ***if, else if, else statements*** there are a few points to keep in mind.

- ❖ An ***if*** can have zero or one else's and it must come after any else if's.
- ❖ An ***if*** can have zero to many else if's and they must come before the else.
- ❖ Once an else if succeeds, none of the remaining else if's or else's will be tested.

Syntax

Following is the syntax of an if...else statement –

```
if(Boolean_expression 1) {  
    // Executes when the Boolean expression 1 is true
```

```
}else if(Boolean_expression 2) {  
    // Executes when the Boolean expression 2 is true  
}else if(Boolean_expression 3) {  
    // Executes when the Boolean expression 3 is true  
}else {  
    // Executes when the none of the above condition is true.}
```

Example

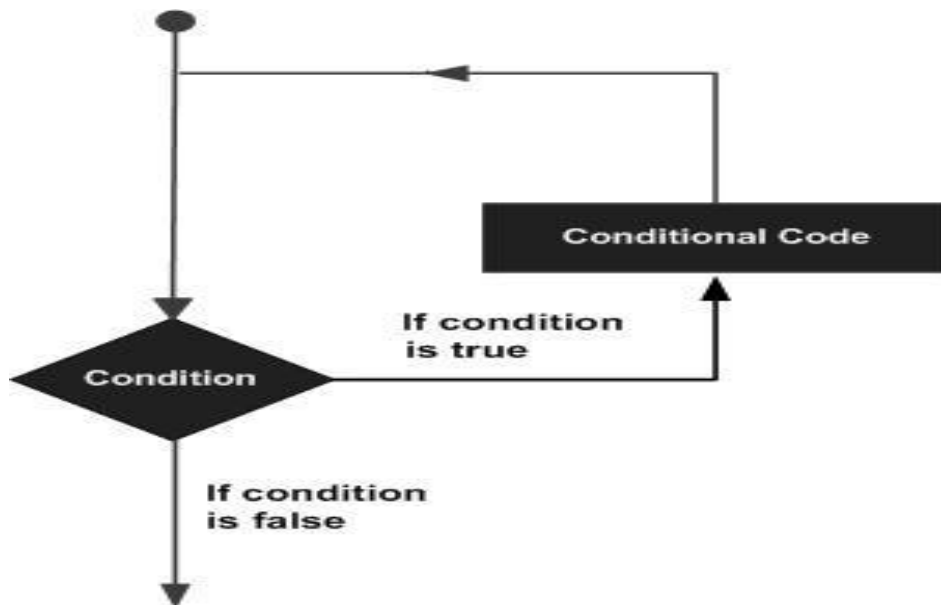
```
Public class Test{  
    Public static void main(Stringargs[]){  
        int x =30;  
        if( x ==10){  
            System.out.print("Value of X is 10");  
        }elseif( x ==20){  
            System.out.print("Value of X is 20");  
        }elseif( x ==30){  
            System.out.print("Value of X is 30");  
        }else{  
            System.out.print("This is else statement");  
        }  
    }  
}
```

This will produce the following result –

OutputValue of X is 30

Loops

Programming languages provide various control structures that allow for more complicated execution paths. A **loop** statement allows us to execute a statement or group of statements multiple times and following is the general form of a loop statement in most of the programming languages.



Object Oriented programming

Java programming language provides the following types of loop to handle looping requirements.

for loop

A **for loop** is a repetition control structure that allows you to efficiently write a loop that needs to be executed a specific number of times.

A for loop is useful when you know how many times a task is to be repeated.

Syntax

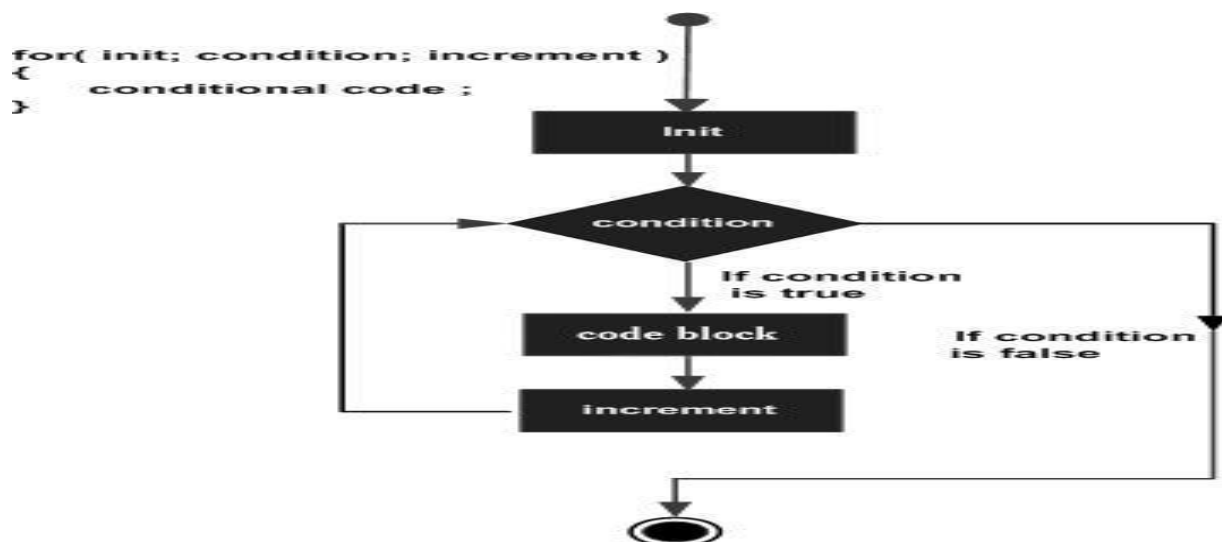
The syntax of a for loop is

```
for(initialization; Boolean_expression; update) {  
    // Statements  
}
```

Here is the flow of control in a **for** loop –

- ❖ The **initialization** step is executed first, and only once. This step allows you to declare and initialize any loop control variables and this step ends with a **semi colon (;)**.
- ❖ Next, the **Boolean expression** is evaluated. If it is true, the body of the loop is executed. If it is false, the body of the loop will not be executed and control jumps to the next statement past the **for loop**.
- ❖ After the **body** of the **for loop** gets executed, the control jumps back up to the **update statement**. This statement allows you to update any loop control variables. This statement can be left blank with a semicolon at the end.
- ❖ The Boolean expression is now evaluated again. If it is true, the loop executes and the process repeats (body of loop, then update step, then Boolean expression). After the Boolean expression is false, the **for loop** terminates.

Flow Diagram



Example

Following is an example code of the for loop in Java.

```
Public class Test{  
Public static void main(String args[]){  
  
for(int x =10; x <20; x = x +1){  
System.out.print("value of x : "+ x );  
System.out.print("\n");  
}}}
```

This will produce the following result –

Output

value of x : 10
value of x : 11
value of x : 12
value of x : 13
value of x : 14

value of x : 15
value of x : 16
value of x : 17
value of x : 18
value of x : 19

while

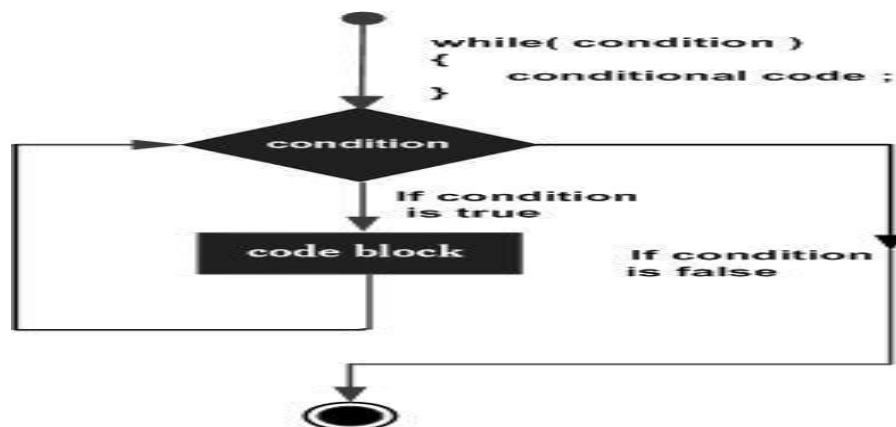
A **while** loop statement in Java programming language repeatedly executes a target statement as long as a given condition is true.

Syntax

The syntax of a while loop is

```
while(Boolean_expression) {  
    // Statements}
```

Here, **statement(s)** may be a single statement or a block of statements. The **condition** may be any expression, and **true** is any **non-zero value**. When executing, if the **boolean_expression** result is true, then the actions inside the loop will be executed. This will continue as long as the expression result is true. When the condition becomes false, program control passes to the line immediately following the loop. **Flow Diagram**



Object Oriented programming

Here, key point of the **while** loop is that the loop might not ever run. When the expression is tested and the result is false, the loop body will be skipped and the first statement after the while loop will be executed.

Example

```
public class Test {  
  
    public static void main(String args[]) {  
        int x = 10;  
  
        while (x < 20) {  
            System.out.print("value of x : " + x);  
            x++;  
            System.out.print("\n");  
        }  
    }  
}
```

This will produce the following result –

Output

```
value of x : 10  
value of x : 11  
value of x : 12  
value of x : 13  
value of x : 14  
value of x : 15  
value of x : 16  
value of x : 17  
value of x : 18  
value of x : 19
```

do...while

A **do...while** loop is similar to a while loop, except that a do...while loop is guaranteed to execute at least one time.

Syntax

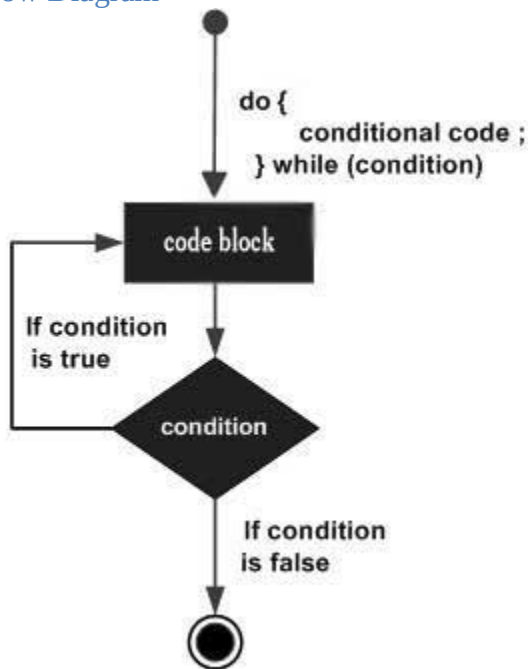
Following is the syntax of a do...while loop –

```
do {  
    // Statements  
} while (Boolean_expression);
```

Notice that the Boolean expression appears at the end of the loop, so the statements in the loop execute once before the Boolean is tested. If the Boolean expression is true, the control jumps back up to do statement, and the statements in the loop execute again. This process repeats until the Boolean expression is false.

Object Oriented programming

Flow Diagram



Example

```
public class Test {  
  
    public static void main(String args[]) {  
        int x = 10;  
        do {  
            System.out.print("value of x : " + x);  
            x++;  
            System.out.print("\n");  
        } while (x < 20);  
    }  
}
```

This will produce the following result –

Output

value of x : 10
value of x : 11
value of x : 12
value of x : 13
value of x : 14
value of x : 15
value of x : 16
value of x : 17
value of x : 18
value of x : 19

break switch

The **break** statement in Java programming language has the following two usages –

Object Oriented programming

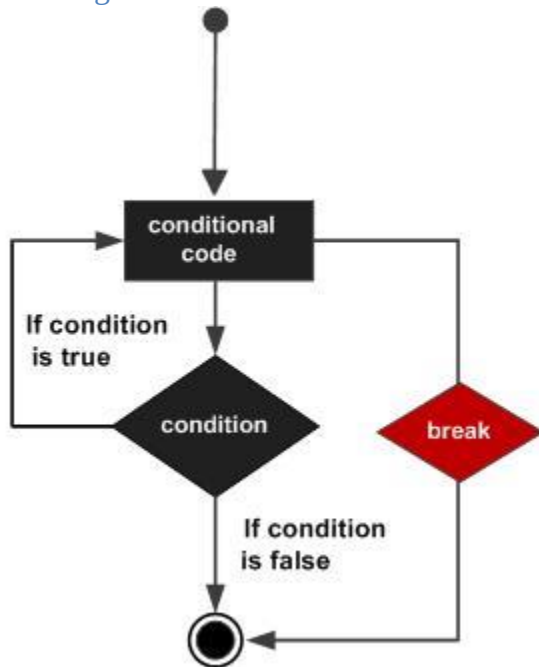
- ❖ When the **break** statement is encountered inside a loop, the loop is immediately terminated and the program control resumes at the next statement following the loop.
- ❖ It can be used to terminate a case in the **switch** statement (covered in the next chapter).

Syntax

The syntax of a break is a single statement inside any loop –

break;

Flow Diagram



Example

```
public class Test {  
  
    public static void main(String args[]) {  
        int[] numbers = {10, 20, 30, 40, 50};  
  
        for (int x : numbers) {  
            if (x == 30) {  
                break;  
            }  
            System.out.print(x);  
            System.out.print("\n=" + x);  
        }  
    }  
}
```

This will produce the following result –

Output

10

20

switch statement

A **switch** statement allows a variable to be tested for equality against a list of values. Each value is called a case, and the variable being switched on is checked for each case.

Syntax

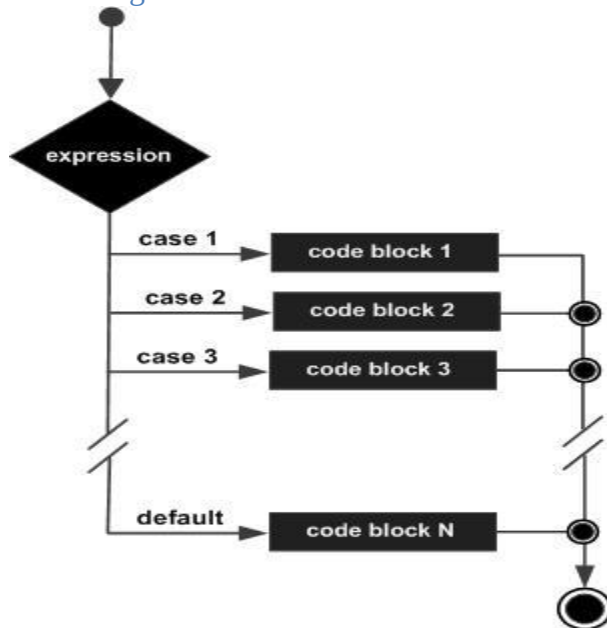
The syntax of enhanced for loop is

```
switch(expression) {  
  case value :  
    // Statements  
  break; // optional  
  
  case value :  
    // Statements  
  break; // optional  
  
  // You can have any number of case statements.  
  default : // Optional  
    // Statements}
```

The following rules apply to a **switch** statement

- The variable used in a switch statement can only be integers, convertible integers (byte, short, char), strings and enums.
- You can have any number of case statements within a switch. Each case is followed by the value to be compared to and a colon.
- The value for a case must be the same data type as the variable in the switch and it must be a constant or a literal.
- When the variable being switched on is equal to a case, the statements following that case will execute until a *break* statement is reached.
- When a *break* statement is reached, the switch terminates, and the flow of control jumps to the next line following the switch statement.
- Not every case needs to contain a break. If no break appears, the flow of control will *fall through* to subsequent cases until a break is reached.
- A *switch* statement can have an optional default case, which must appear at the end of the switch. The default case can be used for performing a task when none of the cases is true. No break is needed in the default case.

Flow Diagram



Example

```
public class Test {  
  
    public static void main(String args[]) {  
        // char grade = args[0].charAt(0);  
        char grade = 'C';  
  
        switch (grade) {  
            case 'A':  
                System.out.println("Excellent!");  
                break;  
            case 'B':  
            case 'C':  
                System.out.println("Well done");  
                break;  
            case 'D':  
                System.out.println("You passed");  
            case 'F':  
                System.out.println("Better try again");  
                break;  
            default:  
                System.out.println("Invalid grade");  
        }  
        System.out.println("Your grade is " + grade);  
    }  
}
```

Compile and run the above program using various command line arguments. This will produce the following result

Output

Well done

Your grade is C

Array

Java provides a data structure, the **array**, which stores a fixed-size sequential collection of elements of the same type. An array is used to store a collection of data, but it is often more useful to think of an array as a collection of variables of the same type. Instead of declaring individual variables, such as ***number0***, ***number1***, ..., and ***number99***, you declare one array variable such as **numbers** and use **numbers[0]**, **numbers[1]**, and ..., **numbers[99]** to represent individual variables. This subtopic introduces how to declare array variables, create arrays, and process arrays using indexed variables.

Declaring Array Variables

To use an array in a program, you must declare a variable to reference the array, and you must specify the type of array the variable can reference. Here is the syntax for declaring an array variable

Syntax

```
dataType [] arrayRefVar; // preferred way.
```

or

```
dataType arrayRefVar[]; // works but not preferred way.
```

Note – The style **dataType[] arrayRefVar** is preferred.

The style **dataTypearrayRefVar[]** comes from the C/C++ language and was adopted in Java to accommodate C/C++ programmers.

Example

The following code snippets are examples of this syntax –

```
double[]myList;// preferred way.
```

or

```
double myList[];// works but not preferred way.
```

Creating Arrays

You can create an array by using the **new** operator with the following syntax –

Syntax

```
arrayRefVar = new dataType[arraySize];
```

The above statement does two things

- ❖ It creates an array using **new dataType[arraySize]**.
- ❖ It assigns the reference of the newly created array to the variable **arrayRefVar**.

Declaring an array variable, creating an array, and assigning the reference of the array to the variable can be combined in one statement, as shown below

```
dataType[] arrayRefVar = new dataType[arraySize];
```

Alternatively you can create arrays as follows –

Object Oriented programming

`dataType[] arrayRefVar = {value0, value1, ..., valuek};`

The array elements are accessed through the **index**. Array indices are 0-based; that is, they start from 0 to **arrayRefVar.length-1**.

Example

Following statement declares an array variable, `myList`, creates an array of 10 elements of double type and assigns its reference to `myList`

```
double[]myList=newdouble[10];
```

Following picture represents array `myList`. Here, `myList` holds ten double values and the indices are from 0 to 9.

Processing Arrays

When processing array elements, we often use either **for** loop or **foreach** loop because all of the elements in an array are of the same type and the size of the array is known.

Example

Here is a complete example showing how to create, initialize, and process arrays –

```
Public classTestArray{
publicstaticvoid main(String[]args){
double[]myList={1.9,2.9,3.4,3.5};
// Print all the array elements
for(int i =0; i <myList.length; i++){
System.out.println(myList[i]+" ");}
// Summing all elements
double total =0;
for(int i =0; i <myList.length; i++){
total+=myList[i];}
System.out.println("Total is "+ total);
// Finding the largest element
double max =myList[0];
for(int i =1; i <myList.length; i++){
if(myList[i]> max)
max =myList[i];}
System.out.println("Max is "+ max);
}}
```

This will produce the following result –

Output

1.9

2.9

3.4

3.5

Total is 11.7

Max is 3.5

The foreach Loops

JDK 1.5 introduced a new for loop known as **foreach** loop or enhanced for loop, which enables you to traverse the complete array sequentially without using an index variable.

Example

The following code displays all the elements in the array myList –

```
public class TestArray {  
  
    public static void main(String[] args) {  
        double[] myList = {1.9, 2.9, 3.4, 3.5};  
  
        // Print all the array elements  
        for (double element : myList) {  
            System.out.println(element);  
        }  
    }  
}
```

This will produce the following result –

Output

```
1.9  
2.9  
3.4  
3.5
```

Passing Arrays to Methods

Just as you can pass primitive type values to methods, you can also pass arrays to methods. For example, the following method displays the elements in an **int** array –

Example

```
public static void printArray(int[] array) {  
    for (int i = 0; i < array.length; i++) {  
        System.out.print(array[i] + " ");  
    }  
}
```

You can invoke it by passing an array. For example, the following statement invokes the printArray method to display 3, 1, 2, 6, 4, and 2 –

Example

```
printArray(new int[] {3, 1, 2, 6, 4, 2});
```

Returning an Array from a Method

A method may also return an array. For example, the following method returns an array that is the reversal of another array –

Example

```
public static int[] reverse(int[] list){
    int[] result = new int[list.length];

    for(int i = 0, j = result.length - 1; i < list.length; i++, j--){
        result[j] = list[i];
    }
    return result;
}
```

Java method

A Java method is a collection of statements that are grouped together to perform an operation/ specific task. When you call the ***System.out.println()*** method, for example, the system actually executes several statements in order to display a message on the console. Now you will learn how to create your own methods with or without return values, invoke a method with or without parameters, and apply method abstraction in the program design.

Creating Method

Considering the following example to explain the syntax of a method –

```
Syntax
public static int methodName(int a, int b) {
    // body goes here
}
```

Now,

- ❖ **public static** – modifier
- ❖ **int** – return type
- ❖ **methodName** – name of the method
- ❖ **a, b** – formal parameters
- ❖ **int a, int b** – list of parameters

Method definition consists of a **method header** and a **method body**. The same is shown in the following syntax

```
Syntax
modifier returnType nameOfMethod (Parameter List) {
    // method body }
}
```

The syntax shown above includes –

- ❖ **modifier** – It defines the access type of the method and it is optional to use.
- ❖ **returnType** – Method may return a value.
- ❖ **nameOfMethod** – This is the method name. The method signature consists of the method name and the parameter list.
- ❖ **Parameter List** – The list of parameters, it is the type, order, and number of parameters of a method. These are optional, method may contain zero parameters.
- ❖ **method body** – The method body defines what the method does with the statements.

Example

Here is the source code of the above defined method called ***minimumFunction()***. This method takes two parameters ***n1*** and ***n2*** and returns the minimum between the two

```
/** the snippet returns the minimum between two numbers */
public static int minimumFunction(int n1, int n2){
    int min;
    if(n1 > n2)
        min = n2;
    else
        min = n1;
    return min;
}
```

Method Calling

For using a method, it should be called. There are two ways in which a method is called i.e., **method returns a value** or **returning nothing (no return value)**. The process of method calling is simple. When a program invokes a method, the program control gets transferred to the called method. This called method then returns control to the caller in two conditions, when:-

- ❖ The return statement is executed.
- ❖ It reaches the method ending closing brace.

The methods returning void is considered as call to a statement. Let's consider an example

```
System.out.println("This is tutorialspoint.com!");
The method returning value can be understood by the following example –
int result = sum(6,9);
```

Following is the example to demonstrate how to define a method and how to call it –

Example

```
public class ExampleMinNumber{
    public static void main(String[] args){
        int a = 11;
        int b = 6;
        int c = minFunction(a, b); //function call
        System.out.println("Minimum Value = " + c);
    }
    /** returns the minimum of two numbers */
    public static int minFunction(int n1, int n2){
        int min;
        if(n1 > n2)
            min = n2;
        else
            min = n1;
        return min;
    }
}
```

This will produce the following result –

Output Minimum value = 6

The void Keyword

The **void** keyword allows us to create methods which do not return a value. Here, in the following example we're considering a void method *methodRankPoints*. This method is a void method, which does not return any value. Call to a void method must be a statement i.e. *methodRankPoints(255.7);*. It is a Java statement which ends with a semicolon as shown in the following example.

Example

```
public class ExampleVoid {  
  
    public static void main(String[] args) {  
        methodRankPoints(255.7);  
    }  
  
    public static void methodRankPoints(double points) {  
        if (points >= 202.5) {  
            System.out.println("Rank:A1");  
        } else if (points >= 122.4) {  
            System.out.println("Rank:A2");  
        } else {  
            System.out.println("Rank:A3");  
        }  
    }  
}
```

This will produce the following result

OutputRank:A1

Passing Parameters by Value

While working under calling process, argument/s is to be passed. These should be in the same order as their respective parameters in the method specification. Parameters can be passed by value or by reference. Passing Parameters by Value means calling a method with a parameter. Through this, the argument value is passed to the parameter.

Example

The following program shows an example of passing parameter by value. The value of the arguments remains the same even after the method invocation.

```
public class swappingExample {  
    public static void main(String[] args) {  
        int a = 30;  
        int b = 45;  
        System.out.println("Before swapping, a = " + a + " and b = " + b);  
        // Invoke the swap method  
        swapFunction(a, b);  
        System.out.println("\n**Now, Before and After swapping values will be same here**");  
        System.out.println("After swapping, a = " + a + " and b is " + b);  
    }  
}
```

```
public static void swapFunction(int a, int b){
    System.out.println("Before swapping(Inside), a = "+ a +" b = "+ b);
    // Swap n1 with n2
    int c = a;
    a = b;
    b = c;
    System.out.println("After swapping(Inside), a = "+ a +" b = "+ b);
}
```

This will produce the following result –

Output

Before swapping, a = 30 and b = 45
Before swapping(Inside), a = 30 b = 45
After swapping(Inside), a = 45 b = 30

*****Now, Before and After swapping values will be same here**:***
After swapping, a = 30 and b is 45

Method Overloading

When a class has two or more methods by the same name but different parameters, it is known as method overloading. It is different from overriding. In overriding, a method has the same method name, type, number of parameters, etc. Let's consider the example discussed earlier for finding minimum numbers of integer type. If, let's say we want to find the minimum number of double type. Then the concept of overloading will be introduced to create two or more methods with the same name but different parameters.

The following example explains the same

Example

```
public class ExampleOverloading{
    public static void main(String[] args){
        int a = 11;
        int b = 6;
        double c = 7.3;
        double d = 9.4;
        int result1 = minFunction(a, b);
        // same function name with different parameters
        double result2 = minFunction(c, d);
        System.out.println("Minimum Value = "+ result1);
        System.out.println("Minimum Value = "+ result2);
    }
    // for integer
    public static int minFunction(int n1, int n2){
        int min;
        if(n1 > n2)
```

```
min= n2;
else
min= n1;
return min;}
// for double
publicstaticdoubleminFunction(double n1,double n2){
double min;
if(n1 > n2)
min= n2;
else
min= n1;

return min;
}}
```

This will produce the following result –

Output

Minimum Value = 6

Minimum Value = 7.3

Overloading methods makes program readable. Here, two methods are given by the same name but with different parameters. The minimum number from integer and double types is the result.

Variable Arguments(var-args)

JDK 1.5 enables you to pass a variable number of arguments of the same type to a method. The parameter in the method is declared as follows –

typeName... parameterName

In the method declaration, you specify the type followed by an ellipsis (...). Only one variable-length parameter may be specified in a method, and this parameter must be the last parameter. Any regular parameters must precede it.

Example

```
publicclassVarargsDemo{

publicstaticvoid main(Stringargs[]){
// Call method with variable args
    printMax(34,3,3,2,56.5);
printMax(newdouble[] {1,2,3});
}

publicstaticvoidprintMax(double... numbers){
if(numbers.length==0){
System.out.println("No argument passed");
}
```

```
return;
}
double result = numbers[0];
for(int i =1; i < numbers.length; i++)
if(numbers[i]> result)
result= numbers[i];
System.out.println("The max value is "+ result);
}}
```

This will produce the following result –

Output

The max value is 56.5

The max value is 3.0

Creating Strings

The most direct way to create a string is to write :-String greeting = "Hello world!";

Whenever it encounters a string literal in your code, the compiler creates a String object with its value in this case, "Hello world!". As with any other object, you can create String objects by using the new keyword and a constructor. The String class has 11 constructors that allow you to provide the initial value of the string using different sources, such as an array of characters.

Example

```
public class StringDemo{

    public static void main(String args[]){
        char[] helloArray={ 'h','e','l','l','o','.' };
        String helloString=new String(helloArray);
        System.out.println(helloString);
    }
}
```

This will produce the following result –

Output

hello

Note – The String class is immutable; so that once it is created a String object cannot be changed.

String Length

Methods used to obtain information about an object are known as **accessor methods**. One accessor method that you can use with strings is the **length()** method, which returns the number of characters contained in the string object. The following program is an example of **length()**, method String class.

Example

Object Oriented programming

```
public class StringDemo {  
    public static void main(String args[]) {  
        String palindrome = "Dot saw I was Tod";  
        int len = palindrome.length();  
        System.out.println("String Length is : " + len);  
    }  
}
```

This will produce the following result –

Output

String Length is : 17

Concatenating Strings

The String class includes a method for concatenating two strings –

```
string1.concat(string2);
```

This returns a new string that is string1 with string2 added to it at the end. You can also use the concat() method with string literals, as in –

```
"My name is ".concat("Zara");
```

Strings are more commonly concatenated with the + operator, as in –

```
"Hello," + " world" + "!"
```

which results in –

```
"Hello, world!"
```

Let us look at the following example –

Example

```
public class StringDemo {  
  
    public static void main(String args[]) {  
        String string1 = "saw I was ";  
        System.out.println("Dot " + string1 + "Tod");  
    }  
}
```

This will produce the following result –

Output

Dot saw I was Tod